Hallucination Analysis in RAG and GraphRAG Systems

Flavien Mathieu
EURECOM
Sophia Antipolis, France
flavien.mathieu@eurecom.fr

Raphael Partouche
EURECOM
Sophia Antipolis, France
raphael.partouche@eurecom.fr

Supervisors: Raphael Troncy, Pasquale Lisena, Carmelle Meli Songuon, Thibault Ehrhart

EURECOM, Sophia Antipolis
Spring 2026

Abstract

Large Language Models augmented with retrieval are increasingly deployed for open-domain question answering, but they remain prone to hallucinations, statements unsupported by the retrieved evidence. This project empirically studies how the retrieval mechanism itself influences the frequency, nature, and types of hallucinations produced by an LLM when answering questions over a fixed knowledge base. We compare classical RAG, which retrieves text chunks by dense vector similarity, with GraphRAG, which retrieves ontology-typed facts from a Knowledge Graph via cosine similarity over verbalised triples. To this end, we enrich two complementary benchmarks, Text2KGBench-LettrIA and OSKGC, with 547 validated question–answer pairs spanning both single-hop and multi-hop reasoning, implement six pipelines sharing the same generator (two baselines, three cross-encoder–reranked hybrids, and a parent–child chunking ablation), and evaluate them using the MIRAGE framework with FactCC alongside a Sentence-BERT correctness metric. Results show that hybrid recall with cross-encoder reranking on the text pipeline yields the highest mean faithfulness and answer correctness, though its faithfulness gain over plain RAG lies within confidence intervals; the statistically significant gap is between the text pipelines and GraphRAG. GraphRAG trails through three distinct failure modes: leaf-node abstention, multi-hop context gaps, and surface-form mismatch with the text-based scorer. A novel dual Graph-Chunk pipeline partially mitigates these failures by fusing text chunks and KG triples, attaining the lowest abstention rate of all pipelines. The objective is not to eliminate hallucinations but to characterise when, why, and how they arise in each paradigm.

Project repository: https://github.com/Mflavien01/GraphRAG-Hallucination-Analysis

1 Introduction

1.1 Motivation

Retrieval-Augmented Generation (RAG) has become a standard architecture for grounding LLM outputs in external knowledge. By injecting passages retrieved from a corpus into the prompt, RAG aims to reduce *parametric hallucinations*—those caused by gaps or noise in the model's pre-trained weights. In practice, however, RAG systems still hallucinate frequently: they may invent entities not present in the retrieved context, infer unsupported relations between real entities, or violate domain constraints. A growing body of work suggests that the underlying *retrieval mechanism*, not just the generator, plays a decisive role in this behaviour. GraphRAG [22] represents a complementary paradigm: instead of retrieving unstructured chunks, it draws its evidence from a structured Knowledge Graph (KG) whose entities and relations are explicitly typed by an *ontology*. The retrieved facts therefore carry explicit relational and type structure rather than being free-form text, which constrains what the generator can assert. The hypothesis motivating this project is that the two paradigms fail differently: RAG should be vulnerable to lexical-semantic confusions (retrieving topically-related but factually-irrelevant chunks), whereas GraphRAG should be vulnerable to ontological gaps, incomplete sub-graphs, and reasoning chains that span several triples.

1.2 Research Question and Roadmap

The central question can be stated as follows: *how do the retrieval mechanisms of RAG and GraphRAG influence the frequency, nature, and typology of hallucinations produced by an LLM?* The objective is not to eliminate hallucinations, but to characterise when, why, and how they appear in each paradigm. The project is structured into three sequential tasks: **(Task 1)** enrich two reference benchmarks with a set of 547 question-answer pairs (200 text-derived via fine-tuned T5, 199 single-hop and 148 multi-hop graph-derived via LLM), produced after a systematic benchmark of five candidate LLMs and a prompt optimisation study; **(Task 2)** implement and compare, on the same source data and the same generator LLM, the RAG and GraphRAG baselines together with three cross-encoder–reranked hybrids (RAG+CE, GraphRAG+CE, and a dual Graph-Chunk+CE) and a parent–child chunking ablation, isolating retrieval as the only variable; **(Task 3)** run all pipelines on the questions of Task 1, evaluate them quantitatively with the MIRAGE framework (FactCC and Sentence-BERT), study their sensitivity to retrieval depth, categorise the hallucinations qualitatively, and contrast the paradigms.

1.3 Contributions

This project makes the following concrete contributions:

- **A balanced evaluation dataset** of 547 question-answer pairs over two KG benchmarks (Text2KGBench-LettriA and OSKGC), produced from 100 proportionally sampled source entries per dataset and unfolded into T5-derived, single-hop, and multi-hop questions with dual text / graph provenance.
- **A systematic LLM benchmark** comparing five candidate models (GPT-4o, Claude Sonnet 4.6, Gemini 2.5 Flash, Mistral 7B, Llama 3.1 8B) on five evaluation criteria to select the best generator for graph-based question generation.
- **A prompt optimisation study** for GPT-4o testing three variants (zero-shot baseline, one-shot with explicit chain format, zero-shot with strict triple format), establishing the importance of the `A -[rel]-> B` chain format for downstream hallucination traceability.
- **Six reproducible QA pipelines** sharing one generator: the RAG and GraphRAG baselines (FAISS over text chunks and over verbalised `A -[rel]-> B` triples respectively); two cross-encoder–reranked hybrids over a FAISS∪BM25 recall pool (RAG+CE, GraphRAG+CE); a novel dual **Graph-Chunk**+CE pipeline that retrieves text chunks and KG triples independently and lets

the generator fuse them through `[KG]`/`[CHUNK]`-labelled context; and a parent–child chunking ablation. An earlier Reciprocal Rank Fusion (RRF) [16] variant is reported as a measured intermediate iteration.

- **A combined hallucination evaluation** using the MIRAGE framework [7] with **FactCC** [15] and a Sentence-BERT correctness metric. Hybrid recall with cross-encoder reranking on the text pipeline (RAG+CE) is the strongest single intervention by mean score: it is the most faithful pipeline (0.560), the best on answer correctness (SBERT 0.464) and on multi-hop questions, and improves over the plain RAG baseline (0.547) on every axis. GraphRAG trails (0.502) through leaf-node abstention, multi-hop context gaps, and surface-form mismatch; the cross-encoder reduces its abstention but not its faithfulness. The dual Graph-Chunk+CE attains the lowest abstention of all pipelines (10.4%) and recovers most of the multi-hop gap without surpassing RAG+CE. This is accompanied by a four-category qualitative taxonomy (parametric memory leakage, surface-form mismatch, multi-hop context gap, and leaf-node abstention).
- **An evaluation-methodology analysis.** We surface and correct a score-orientation inversion in the MIRAGE/FactCC interface (faithfulness $= 1 -$ score, validated against SBERT), document FactCC's surface-form bias against ontology-derived answers, show that FactAcc (REBEL-based) is unsuitable for short KG answers, and report a systematic retrieval-depth (k) study revealing that text pipelines saturate at small k while graph pipelines require larger k.

AI usage disclosure. A large language model (GPT-4o) was used as the generator in Task 1 for graph-based question generation. Its use is transparently described in Section 4 and constitutes part of the experimental methodology.

2 Related work

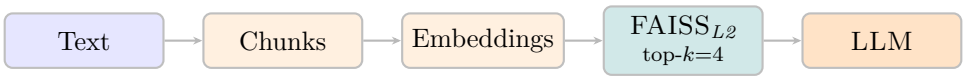
RAG, KGs and GraphRAG. The two paradigms compared in this project share the same generator stage but differ fundamentally in how they retrieve evidence (Figure 1).

RAG relies on *semantic-distributional* retrieval: chunks of the source corpus are encoded into dense vectors and the top-k closest to the query embedding are injected into the prompt. This is robust to surface variations but blind to logical structure, a chunk that mentions the right entities in the wrong relation can rank highly.

KGs and ontologies. A Knowledge Graph [23] is a database of (*subject*, *predicate*, *object*) triples constrained by an *ontology*, which defines the admissible entity types, the admissible relations with their explicit domain and range, and a subsumption hierarchy such as `President`, `Politician` or `Person`. A triple violating the ontology is therefore not merely unlikely but *ill-formed*.

GraphRAG replaces the vector retriever by a structured one operating on the KG: each triple `(subject, predicate, object)` is verbalised into a natural-language statement, embedded with the same encoder as the RAG pipeline, and the top-k most relevant statements are retrieved by cosine similarity against the query. The structured, ontology-constrained origin of these facts is preserved in the context injected into the prompt, which is the key distinction from classical RAG.

Hybrid retrieval and reranking. Dense bi-encoder retrieval captures semantic similarity but can miss exact lexical matches such as rare named entities, identifiers, or numerical values. Sparse lexical retrieval such as BM25 [19] is complementary, rewarding exact term overlap. The two signals are classically combined by *Reciprocal Rank Fusion* (RRF) [16], which merges ranked lists using only the rank positions of each retriever, without reading the passages themselves. A stronger alternative is a *cross-encoder* reranker: whereas a bi-encoder embeds the query and each passage independently, a cross-encoder concatenates the (query, passage) pair and passes it jointly through a transformer, capturing fine-grained interactions at a higher inference cost [18]. This *recall-and-rerank* pattern, in which a cheap retriever produces a candidate pool that an expensive reranker then reorders, directly motivates the hybrid pipelines of Section 5, where RRF over a FAISS∪BM25 pool is superseded by a cross-encoder.



(a) Classical RAG: `IndexFlatL2` search over text chunks.

(b) GraphRAG: `IndexFlatIP` cosine search over verbalised KG triples (`A-[rel]->B` format).

Figure 1: The two retrieval paradigms. Both share the same generator (LLM) and encoder (`all-MiniLM-L6-v2`); they differ only in what is indexed: raw text chunks (RAG) versus verbalised KG triples (GraphRAG). The retrieval depths shown (k=4 and k=12) are the per-paradigm optima selected by the K-sweep of Section 6.11.

Hallucinations. We call *hallucination* any statement in the model's answer that is not entailed by the retrieved evidence. Following the typology adopted in the project description, we distinguish four *a-priori* categories:

- **Invented entities** : entities mentioned in the answer but absent from the retrieved context.
- **Incorrect relations** : existing entities linked by an unsupported relation.
- **Unsupported facts** : assertions drawn from the model's parametric memory rather than from the retrieved context.
- **Ontology violations** : assertions whose subject or object types are incompatible with the predicate's domain or range.

The two latter categories are particularly relevant for the comparison: GraphRAG is, in principle, the only paradigm in which an ontology violation can be defined and measured, since classical RAG has no symbolic schema to violate. In Section 6.7 we complement this a-priori typology with the failure modes actually observed in the pipeline outputs (parametric memory leakage, surface-form mismatch, multi-hop context gaps, and leaf-node abstention) and relate the two. The quantitative scoring in Task 3 relies on the MIRAGE framework [7].

Question generation from text and graphs. Automatic question generation has been studied both from unstructured text and from structured KGs. QGwSRL [6] frames question generation as a sequence-to-sequence task guided by Semantic Role Labelling, producing questions whose argument structure is explicitly grounded in the source sentence. KGEL [5] takes the complementary approach of traversing KG triples to generate multi-hop questions that require chaining two or more facts. In practice, fine-tuned `T5`-based models such as `valhalla/t5-small-qa-qg-hl` [3] offer a lightweight and reproducible alternative for text-based generation on domain-specific corpora.

Few-shot prompting and LLM-as-generator. Brown et al. [9] demonstrated that large language models can perform new tasks from a small number of in-context examples without any weight update. This *few-shot* capability underpins the LLM-as-generator approach adopted in Task 1: by providing a single labelled example of the desired `A-[rel]->B` chain format, the prompt anchors the model to the structured output without fine-tuning, while negative grounding rules suppress parametric hallucinations at generation time.

3 Datasets

The project uses two complementary benchmarks presented at ISWC 2025 within the joint proceedings of KBC-LM and LM-KBC. Both ultimately derive from the WebNLG corpus and its DBpedia-derived ontology, but differ in scope and granularity.

Text2KGBench-LettrIA [1] is a refined revision of Text2KGBench produced by LettrIA and EURECOM. It provides **19 domain ontologies** with a hierarchical structure and formal typing, and re-annotates 4 860 **sentences** (2 012 in the development split used in this project) into roughly 14 000 high-fidelity triples.

OSKGC [2] (*Ontology Schema-based Knowledge Graph Construction*) places the ontology schema at the centre of the task. It provides 10 183 text entries split into 19 thematic categories and three triple-count groups (1, 2, or 3 triples per entry), yielding **57 sub-categories**, with **207 entity types**, **382 relations**, and a subsumption hierarchy of maximum depth 4 derived from DBpedia. Each entry is stored as an XML block pairing the text with its triples and their type-level schemas.

Why two datasets? The two benchmarks are complementary along three axes (Table 1): Text2KGBench-LettrIA is denser per category but uses flatter ontologies, while OSKGC provides finer-grained types and a deeper subsumption hierarchy. Combining them broadens the domain and ontology coverage of the evaluation set and reduces the risk that the hallucination behaviour observed in Task 3 is an artefact of a single benchmark's annotation style or ontology design.

Table 1: High-level comparison of the two reference datasets used in the project.

	Text2KGBench-LettrIA	**OSKGC**
Source corpus	DBpedia-WebNLG	WebNLG
Top-level / sub-categories	19	19 / 57 (split by triple count)
Annotated sentences	4 860	10 183
Ontology design	Hierarchical, strict typing	Hierarchical, max depth 4
Property types	Object vs Datatype properties	Single relation type
Storage format	JSONL per category	XML per sub-category
Dedicated metric	Hierarchical precision/recall	Structural Similarity (SS)

4 Task 1 – Dataset Enrichment with Questions

The objective of Task 1 is to extend each benchmark with a controlled set of question-answer (QA) pairs that will, in Task 3, drive the comparative evaluation of the question-answering pipelines. Two design constraints structured the methodology. First, **balanced complexity**: the question set must contain both *single-hop* questions (answerable from a single triple) and *multi-hop* questions (requiring traversal of two or more connected triples), in order to expose the divergence between RAG and GraphRAG, vector retrieval is expected to behave reasonably on single-hop queries but to degrade on multi-hop ones. Second, **balanced provenance**: questions are generated both from the source text (reproducing what a reader of the original sentence would naturally ask) and from the KG (reproducing what a graph traversal naturally exposes), so that neither retrieval paradigm is favoured by the question distribution.

4.1 Statistical Analysis and Proportional Sampling

Both benchmarks are heavily imbalanced across categories (Tables 2 and 3). For example, in Text2KGBench-LettrIA, the `city` category accounts for 10.8% of all sentences while `monument` accounts for only 0.9%. A naive uniform sampling would under-represent the dominant categories and over-represent the rare ones, distorting the distributional properties any retriever would meet in practice. We therefore implemented a **proportional stratified sampler**: for a target sample of n entries, each category contributes a number of entries proportional to its share of the total, with a floor of one entry per category to preserve coverage; sampling is performed without replacement, with a fixed seed for reproducibility, and the final list is shuffled to mix categories. We set $n = 100$ for both datasets independently, yielding 100 source entries per dataset (200 total) before question unfolding.

Table 2: Text2KGBench-LettrIA development split. Percentages drive the sampler.

Category	Sent.	Trip.	%
airport	79	260	3.9
artist	84	256	4.2
astronaut	68	266	3.4
athlete	107	304	5.3
building	102	276	5.1
celestialbody	71	203	3.5
city	217	1289	10.8
comicscharacter	36	92	1.8
company	56	174	2.8
film	127	368	6.3
food	153	473	7.6
meanoftransportation	92	271	4.6
monument	19	64	0.9
musicalwork	209	842	10.4
politician	135	415	6.7
scientist	149	387	7.4
sportsteam	110	375	5.5
university	71	337	3.5
writtenwork	127	267	6.3
TOTAL	**2012**	**6919**	**100.0**

Table 3: OSKGC development split, aggregated across the three triple-count groups.

Category	Sent.	Trip.	%
Airport	81	149	8.2
Artist	88	173	8.9
Astronaut	22	44	2.2
Athlete	78	149	7.9
Building	65	126	6.6
CelestialBody	49	94	5.0
City	74	145	7.5
ComicsCharacter	27	50	2.7
Company	29	57	2.9
Film	16	30	1.6
Food	86	176	8.7
MeanOfTransportation	81	158	8.2
Monument	10	21	1.0
MusicalWork	17	31	1.7
Politician	90	176	9.1
Scientist	17	32	1.7
SportsTeam	66	123	6.7
University	19	38	1.9
WrittenWork	75	152	7.6
TOTAL	**990**	**1919**	**100.0**

4.2 LLM Selection: Benchmark and Prompt Optimisation

Before running the full generation pipeline, we conducted two preliminary studies to select the most appropriate model and prompt for graph-based question generation.

Model benchmark. Five candidate LLMs were evaluated on a fixed sample of KG triples drawn from both datasets: GPT-4o, Claude Sonnet 4.6, Gemini 2.5 Flash, Mistral 7B, and Llama 3.1 8B. Each model was scored manually on five criteria: (1) grammatical correctness, (2) semantic faithfulness to the input triple, (3) natural-language fluency, (4) visibility of relation names in the output chain, and (5) multi-hop chain coherence. GPT-4o was selected as the final generator: the prompt optimisation study described below confirmed its superior chain traceability on criteria (4) and (5), which is essential for the downstream hallucination analysis. Gemini 2.5 Flash was considered as a free-tier alternative but produced less consistent relation chains.

Prompt optimisation. Three prompt variants were tested on GPT-4o at temperature 0 to identify the design most suitable for downstream hallucination traceability:

- **Variant A** (zero-shot baseline): a plain instruction with no format constraint, producing free-form natural-language questions with no explicit chain.
- **Variant B** (one-shot with explicit chain format): a single labelled example demonstrating the `entity1 -[rel]-> entity2 -[rel]-> answer` structure. *Selected as the final prompt.*
- **Variant C** (zero-shot with strict triple format): a structured output constraint without any example, producing well-formed JSON but with less consistent chain visibility.

Variant B was selected because it consistently produces explicit `A -[rel]-> B` chains with visible relation names. This chain format is not merely a cosmetic preference: it provides a direct audit trail from each generated question back to the originating triples, which is essential for the hallucination traceability analysis of Task 3. This design choice is grounded in the few-shot prompting literature: Brown et al. [9] showed that a single in-context example significantly reduces output variance and anchors the model to the desired format without any weight update. Prompts were submitted to the project supervisors for validation before running the full generation, to avoid unnecessary API costs.

4.3 Question Generation

The 200 sampled entries are fed in parallel into two complementary generators. Each entry produces both a text-derived QA pair and one or two graph-derived QA pairs, so that both provenances are well represented in the final set (200 text-derived T5 questions and 347 graph-derived single- and multi-hop questions).

Text-based generation: fine-tuned T5. For text-based question generation we use `valhalla/t5-small-qa-qg-hl` [3], a `t5-small` model fine-tuned on SQuAD and distributed through the `patil-suraj/question_generation` repository [4]. The pipeline is two-step. First, *answer extraction*: the sentence is split on punctuation and each clause is passed to the model with the prefix `extract_answers:` surrounded by `<hl>` sentinels; the model returns a `<sep>`-separated list of candidate answer spans, of which we retain only those that literally appear in the original sentence; this filtering already discards the most overt T5 hallucinations. Second, *question generation*: for each retained answer, the original sentence is re-emitted with the answer wrapped in `<hl>` markers and prefixed by `generate question:`, and the model produces a natural-language question whose expected answer is the highlighted span.

Graph-based generation: strictly-prompted LLM. For graph-based generation, rather than reusing a specialised model that operates on text such as KGEL [5], we adopt the *LLM-as-generator* option explicitly listed as an admissible alternative in the project plan. The rationale is twofold: the KGEL reference implementation is sparsely documented and not plug-and-play on the two benchmarks at hand, and the typed triples of Text2KGBench-LettrIA and OSKGC provide a sufficiently constrained input that an LLM can be prompted to behave as a deterministic graph-walker, provided the prompt enforces strict grounding rules. We use **GPT-4o** as the underlying LLM, accessed through the OpenAI API at temperature 0 for deterministic, reproducible generation. Before prompting, the generator runs a *multi-hop feasibility check*: two triples are deemed connected if they share at least one entity, and if no two triples are connected the prompt forbids the multi-hop branch entirely (its slot is filled with a canonical `insufficient_connected_triples` placeholder), so the model is never forced to fabricate a fake chain. The prompt itself is a single-shot structured instruction with three notable design decisions: a fixed JSON output schema (which keeps downstream parsing trivial), grounding rules stated as *strict negatives* ("do NOT invent…", "do NOT use world knowledge…")—empirically more effective than positive instructions alone—and, for the multi-hop branch, an explicit `chain` field of the form `entity1 -[rel]-> entity2 -[rel]-> answer` that serves as both self-explanation and audit trail for manual validation. The full code, including the retry layer that handles JSON parsing errors and transient API failures, is available in the project repository.

4.4 Output and Manual Validation

The two pipelines produce four JSONL files (one per dataset and per generator), each line being a self-contained record carrying the originating sentence, the originating triples, and the generated question(s)—all the information needed to feed the RAG and GraphRAG pipelines of Task 2 without any further joining. The dual provenance and the explicit single-vs-multi-hop labelling will allow Task 3 to slice the hallucination analysis along three orthogonal axes: retrieval paradigm (RAG vs GraphRAG), question source (text- vs graph-derived), and question complexity (single- vs multi-hop).

Both generators have known limitations that required a systematic manual review of the output. The T5 model is small (60M parameters) and occasionally produces disfluent questions or selects trivial answer spans (e.g. a single preposition); more importantly, its fine-tuning on SQuAD biases it towards *simple, single-step* questions that target one explicit fact of the input sentence. To restore the complexity balance targeted in Section 4, we manually reformulated around 30% of the T5-generated questions into more compositional variants that require chaining two facts contained in the same sentence. For example, given the sentence "Camilla was discovered by the British N. R. Pogson, who later died in Chennai", the T5 model naturally produces "Who discovered Camilla?"; we reformulated it into "Where did the person who discovered Camilla die?", which forces the answering system to first identify the discoverer and then retrieve the death location. On the LLM side, despite the explicit negative instructions, residual world knowledge can still leak into the model's phrasings (e.g. "the famous capital of…"). This is a known failure mode of LLM-based question generation and is the very phenomenon Task 3 is designed to study, so we treat it as a feature of the experimental setup rather than a bug, with the proviso that the questions themselves must remain answerable from the local context.

For these reasons, every generated QA pair was **manually reviewed**. We spot-checked fluency, grounding, and the validity of the reasoning chains; discarded any question that could not be answered from its associated triples or sentence, corrected minor surface errors when the underlying intent was clearly recoverable, and applied the compositional reformulation described above to a fraction of the T5 output. The resulting validated question set becomes the evaluation input of the retrieval pipelines.

5 Task 2 – Question-Answering Pipelines

The objective of Task 2 is to implement question-answering pipelines that operate on identical input data — the source corpora of Text2KGBench-LettrIA and OSKGC — and are evaluated on the same question set produced in Task 1. The variable between the two baseline pipelines is the *retrieval mechanism*: dense vector similarity over text chunks for RAG, and cosine similarity over verbalised KG triples for GraphRAG. Three further pipelines extend the baselines with a hybrid **Cross-Encoder** recall-and-rerank stage (RAG+CE, GraphRAG+CE, and the dual Graph-Chunk+CE), and a parent–child chunking variant is evaluated as an ablation. All pipelines share the same generator LLM, ensuring that any observed difference in hallucination behaviour is attributable to retrieval, not generation.

5.1 Shared Design Principles

Three design decisions are common to all pipelines and are fixed for the entire evaluation.

Same generator LLM. All pipelines call the same language model: **Qwen3-30B-A3B-Instruct-2507** [12], a Mixture-of-Experts model with 30 B total parameters and 3 B active parameters at inference time, accessed via the EURECOM self-hosted LiteLLM gateway. The LLM interface is abstracted behind a thin wrapper (`llm/llm_interface.py`) that decouples the retrieval logic from the generation backend.

Same input data. All pipelines index the same source corpora: the raw sentences from Text2KGBench-LettrIA (stored as JSONL) and from OSKGC (stored as XML). The RAG, RAG+CE and parent–child pipelines operate on the text form of these entries; GraphRAG and GraphRAG+CE operate on the verbalised triples; and Graph-Chunk+CE uses both. No additional KG extraction step is required since both benchmarks already provide typed triples alongside the source sentences.

Fixed evaluation protocol. Each pipeline receives the 547 validated questions of Task 1 (200 T5-derived, 199 single-hop and 148 multi-hop LLM-derived), runs retrieval and generation for each question, and writes its outputs to a JSONL file. The orchestration script implements **checkpoint/resume** (completed `(id, hop_type)` pairs are skipped on re-runs) and **exponential retry** with jitter (up to 300 s) to handle transient API failures.

5.2 RAG Pipeline

The classical RAG pipeline follows the standard *index-then-retrieve* architecture [8].

Chunking. Source sentences are split into overlapping text windows using `RecursiveCharacterTextSplitter` (`LangChain` [13]), with a chunk size of 500 characters and an overlap of 50. Overlapping chunks ensure that facts spanning a sentence boundary are not cut off.

Embedding. Each chunk is encoded with `all-MiniLM-L6-v2` (`sentence-transformers`), a lightweight model that runs locally with no API cost.

Vector store. Embeddings are indexed in **FAISS** [10] (`IndexFlatL2`). We chose FAISS over server-based alternatives because the project operates on a fixed corpus with no need for cross-session persistence.

Retrieval and generation. At query time, the top-k (k=4) nearest chunks are retrieved by L2 distance. The retrieved chunks and the question are assembled into a prompt and passed to the generator via the `EurecomLLM` wrapper. The choice k=4 is the optimum identified by the retrieval-depth study of Section 6.11.

$$\textit{Text} \rightarrow \textit{Chunks} \rightarrow \textit{Embeddings} \rightarrow \text{FAISS}_{\text{L2}} \xrightarrow{k=4} \textit{LLM} \rightarrow \textit{Answer}$$

5.3 GraphRAG Pipeline

The GraphRAG pipeline replaces the text-chunk retriever by one operating on KG triples.

Initial design: entity linking by approximate string matching. The initial implementation identified anchor nodes via n-gram matching (**RapidFuzz** [11], threshold 95) followed by BFS traversal. This exhibited a fundamental structural limitation: when the matched anchor was a *leaf node* (no outgoing edges), the traversal returned empty context, causing the generator to hallucinate.

Revised design: statement-level cosine similarity. Following supervisor feedback, retrieval was revised to the *verbalised triple* level. Each triple `(subject, predicate, object)` is serialised as `subject -[predicate]-> object` and embedded with the same `all-MiniLM-L6-v2` encoder. The top-k (k=12) statements are retrieved by cosine similarity (FAISS `IndexFlatIP` with normalised embeddings); k=12 is the optimum from Section 6.11, notably larger than RAG's k=4 because each triple carries a single atomic fact.

$$\textit{Triples} \rightarrow \textit{Verbalise} \rightarrow \textit{Embed} \rightarrow \text{FAISS}_{\text{IP}} \xrightarrow{k=12} \textit{Context} \rightarrow \textit{LLM} \rightarrow \textit{Answer}$$

This revision resolves the leaf-node failure mode: every triple is a candidate regardless of its topological position. However, it introduces a new limitation for multi-hop queries: cosine similarity over individual triples cannot recover a two-hop reasoning chain in a single retrieval pass.

5.4 Hybrid Recall and Cross-Encoder Reranking

Analysis of the baseline outputs in Task 3 revealed that both pipelines suffer from *retrieval noise*: passages retrieved by the bi-encoder are semantically related to the question but do not necessarily contain the answer, and exact lexical matches (rare named entities, identifiers, numbers) are sometimes missed entirely. The three +CE pipelines address this with a two-stage *recall-and-rerank* retriever.

Stage 1 — hybrid recall (FAISS ∪ BM25). To maximise the chance that the answer-bearing evidence reaches the reranker, the candidate set is widened beyond pure dense retrieval. A sparse BM25 [19] retriever runs alongside the dense FAISS retriever; each returns its top $6k$ candidates and their *union* forms the candidate pool. BM25 contributes exact lexical matches that the dense encoder ranks poorly, while FAISS contributes semantically similar but lexically divergent evidence.

Stage 2 — cross-encoder reranking. The pooled candidates are then rescored by a **Cross-Encoder**. Unlike the bi-encoder used in retrieval (which embeds the question and each passage independently), a cross-encoder concatenates the (question, passage) pair and passes the full pair through a transformer encoder, producing a single relevance score; this captures fine-grained interactions between the two inputs at the cost of higher inference time, acceptable on our fixed evaluation corpus. We use `cross-encoder/ms-marco-MiniLM-L-6-v2` from `sentence-transformers`, fine-tuned on the MS MARCO passage-ranking task, which requires no additional training on our data. The reranker returns the top k candidates as the final context.

From RRF to cross-encoder: a measured iteration. The hybrid pool was *initially* fused with **Reciprocal Rank Fusion** (RRF) [16], which merges the FAISS and BM25 rank lists using only rank positions, without reading the passages. RRF improved recall but, being content-blind, let semantically off-topic passages through and faithfulness stagnated. We therefore replaced RRF by the cross-encoder. The change was measured directly: on the Graph-Chunk pipeline, the RRF variant scored a faithfulness of 0.533 at a 13.5% coverage-error rate, which the cross-encoder improved to 0.540 at 11.0%. RRF is therefore reported as a measured intermediate step rather than the final design.

Implementation. The hybrid retriever is implemented in `rag/vector_store.py` (`search_hybrid`) for the text-chunk side and in `graph_rag/statement_retriever_hybrid.py` for the triple side, and is shared across the +CE pipelines through a common interface. It is applied after the FAISS ∪ BM25 recall stage and before context serialisation.

5.5 Hybrid Pipelines

RAG + Cross-Encoder. The text-chunk pipeline with the hybrid recall-and-rerank stage of Section 5.4 (k=4). Results in Task 3 show that the reranker improves the RAG baseline on every axis: faithfulness ($\Delta = +0.013$, $0.547 \rightarrow 0.560$), answer correctness (SBERT $\Delta = +0.018$), multi-hop faithfulness ($\Delta = +0.024$), and coverage errors ($17.2\% \rightarrow 11.7\%$). RAG+CE is in fact the most faithful pipeline of the study: hybrid recall surfaces lexical matches the dense retriever missed, and the cross-encoder reorders them so the answer-bearing passage reaches the generator.

GraphRAG + Cross-Encoder. The triple pipeline with the same hybrid recall-and-rerank stage (k=13). The cross-encoder helps GraphRAG on abstention and correctness but *not* on faithfulness: coverage errors drop from 33.5% to 27.4% and SBERT rises by $\Delta = +0.027$ ($0.410 \rightarrow 0.437$), while faithfulness is flat-to-slightly-lower ($0.502 \rightarrow 0.499$, $\Delta = -0.004$). Contrasted with RAG+CE, this shows the reranker's benefit is concentrated on the text modality; on triple retrieval its main contribution is reducing leaf-node abstention.

$$\textit{Triples} \rightarrow \textit{Embed} \rightarrow (\mathrm{FAISS}_{\mathrm{IP}} \cup \mathrm{BM25}) \xrightarrow{6k} \textit{pool} \rightarrow \textit{Cross-Encoder} \xrightarrow{k} \textit{LLM} \rightarrow \textit{Answer}$$

Graph-Chunk + Cross-Encoder. Graph-Chunk+CE is a novel dual hybrid. Crucially, it does *not* merge text chunks and triples into a single pool: it runs *two independent* recall-and-rerank retrievers — the text-chunk retriever ($\mathrm{FAISS}_{\mathrm{L2}} \cup \mathrm{BM25}$, reranked by the cross-encoder) and the triple retriever ($\mathrm{FAISS}_{\mathrm{IP}} \cup \mathrm{BM25}$, reranked by the cross-encoder) — under a *fixed budget* (k_{chunk}=20 chunks and k_{graph}=20 triples). The two reranked lists are then *concatenated*, each block tagged with a `[CHUNK]` or `[KG]` label, and the *generator itself* performs the final fusion by attending to both labelled sources. The fusion is therefore performed by the LLM at generation time, not by the retrieval algorithm. This design is implemented in `graph_rag/pipeline_graph_chunk.py`.

$$\begin{aligned}
\text{chunk leg:}\quad & \textit{Chunks} \rightarrow (\mathrm{FAISS}_{\mathrm{L2}} \cup \mathrm{BM25}) \rightarrow \textit{CE} \xrightarrow{k_{\mathrm{chunk}}} C_{\texttt{[CHUNK]}} \\
\text{triple leg:}\quad & \textit{Triples} \rightarrow (\mathrm{FAISS}_{\mathrm{IP}} \cup \mathrm{BM25}) \rightarrow \textit{CE} \xrightarrow{k_{\mathrm{graph}}} C_{\texttt{[KG]}} \\
& (C_{\texttt{[CHUNK]}} \,\|\, C_{\texttt{[KG]}}) \rightarrow \textit{LLM} \rightarrow \textit{Answer}
\end{aligned}$$

The intuition is that text chunks provide lexical coverage (named entities and numerical values stated verbatim) while verbalised triples provide relational structure (explicit predicate typing); keeping the budgets fixed guarantees that both modalities are always present, and the `[KG]` / `[CHUNK]` labels let the generator weigh them. Graph-Chunk+CE achieves the lowest coverage-error rate of all pipelines (10.4%) and recovers most of GraphRAG's multi-hop deficit (multi-hop faithfulness

0.426, $\Delta = +0.113$ over the GraphRAG baseline and above the plain RAG baseline's 0.414). It does not, however, surpass RAG+CE on any global metric.

Table 4 summarises all six pipelines.

Table 4: Summary of the six evaluation pipelines and the retrieval depth k selected by the K-sweep (Section 6.11).

Pipeline	Retriever	Reranker	k	Output file
RAG	FAISS L2 (chunks)	—	4	`rag_results.jsonl`
GraphRAG	FAISS IP (triples)	—	12	`graphrag_results.jsonl`
RAG + CE	FAISS L2 ∪ BM25 (chunks)	Cross-Encoder	4	`rag_hybrid_results.jsonl`
GraphRAG + CE	FAISS IP ∪ BM25 (triples)	Cross-Encoder	13	`graphrag_hybrid_results.jsonl`
Graph-Chunk + CE	both hybrid retrievers	Cross-Encoder	20+20	`graph_chunk_results.jsonl`
RAG Parent-Child	FAISS L2 (child→parent)	—	4	`rag_parent_child_results.jsonl`

5.6 Parent-Child Chunking

The pipelines above use a *flat* chunking strategy: each source sentence is split into fixed-size overlapping windows, and all chunks are treated as equal candidates during retrieval. This strategy has a known limitation: when the relevant fact spans a sentence boundary, the overlap may capture it partially, but the broader discourse context (the surrounding sentences that establish the entity being discussed) is absent from the retrieved chunk.

Motivation. **Parent-child chunking** [14] addresses this limitation by maintaining two granularities simultaneously. Small *child chunks* (fine-grained) are used for retrieval: their high specificity makes them more likely to match the query semantically. When a child chunk is selected, its *parent chunk* (the larger surrounding passage) is injected into the context instead. The retrieval step thus benefits from the precision of fine-grained indexing while the generator receives wider discourse context. This design is particularly motivated by the **parametric memory leakage** failure mode of Section 6.7: when the LLM cannot find the answer in a narrow chunk, it falls back on parametric memory, and a wider parent context should reduce that fallback.

Implementation. Parent-child chunking is implemented in `rag/chunker_parent_child.py`. Each source text is split into parent chunks of size c_{parent} and each parent is further split into child chunks of size c_{child} ($c_{\text{child}} < c_{\text{parent}}$). Child embeddings are indexed in FAISS; a lookup table maps each child identifier to its parent text. At query time the top $4k$ child chunks are retrieved, deduplicated to their k=4 distinct parents, and the parent texts are injected as context. The configuration used is $c_{\text{parent}} = 1500$, $c_{\text{child}} = 150$, with an overlap of 20 characters at both levels.

Results. Table 5 reports the parent-child variant against the flat RAG baseline and Graph-Chunk+CE. Contrary to the motivating hypothesis, parent-child chunking does not improve grounding over the flat RAG baseline: faithfulness is slightly lower (0.534 vs. 0.547, $\Delta = -0.013$) and answer correctness is essentially unchanged (SBERT 0.445 vs. 0.446), although abstention is somewhat reduced (15.2% vs. 17.2%). The faithfulness loss is concentrated on T5 questions (0.605 vs. 0.636, $\Delta = -0.031$): these are answerable from a single sentence, so injecting a large 1500-character parent block surrounds the relevant sentence with unrelated neighbouring text and dilutes the evidence the generator must attend to. This mirrors the retrieval-depth analysis of Section 6.11, where text pipelines lose faithfulness as more context is added.

The result is neutral-to-negative: widening the textual context trades a little faithfulness for a little less abstention, and in particular it does not reach RAG+CE, which improves faithfulness, correctness and abstention simultaneously. What reduces hallucination here is *reranked, filtered* evidence (RAG+CE, Graph-Chunk+CE), not merely more surrounding text.

Table 5: Parent-child chunking vs. baseline pipelines (coverage errors excluded; faithfulness on answered entries only).

Pipeline	Answered	Coverage err.	Mean faith.	Halluc.	SBERT
RAG baseline	453 / 547	17.2%	**0.547**	**44.6%**	0.446
Graph-Chunk + CE	490 / 547	**10.4%**	0.527	47.1%	**0.454**
RAG Parent-Child	464 / 547	15.2%	0.534	46.1%	0.445

6 Task 3 – Hallucination Evaluation

6.1 Evaluation Framework

We evaluate factual consistency using the MIRAGE library [7], which provides a standardised interface to multiple factuality metrics. We selected **FactCC** [15], a BERT-based classifier fine-tuned to predict whether a generated statement is supported by a given source document. For each question–answer pair, FactCC receives the *original source sentence from the dataset* as its document input and the LLM-generated answer as the candidate statement.

Score direction. The MIRAGE library exports $P(\text{INCORRECT})$ as its `score` field. We therefore compute:

$$\text{faithfulness} = 1 - \text{score}$$

so that higher values indicate greater consistency with the source. This correction was identified via a diagnostic cross-check: the Pearson correlation between faithfulness and Sentence-BERT cosine similarity (§6.6) flips sign once the orientation is corrected — from clearly negative ($r \approx -0.24$) to clearly positive ($r \approx +0.24$) — confirming that the corrected orientation is the meaningful one. We define a *hallucination event* as any answered entry with faithfulness < 0.5.

Coverage errors. We distinguish *coverage errors* — cases where the pipeline produces no usable answer (typically an explicit abstention such as *"the context does not contain information about X"*) — from genuine hallucination attempts. Coverage errors are excluded from the faithfulness calculation and reported separately as a *coverage error rate*. This separation is important because GraphRAG's higher abstention rate on leaf-node queries would otherwise inflate its apparent hallucination rate when aggregated with hallucination events. Operationally, a response is classified as a coverage error by a *regular-expression detector* (`is_coverage_error`) that matches explicit abstention patterns in the generated answer — e.g. "the context does not contain", "cannot be determined", "I cannot answer" — rather than by thresholding the FactCC score. Detecting abstention from the answer text keeps the decision independent of the faithfulness metric under evaluation.

6.2 Metric Selection and Evaluation Challenges

Domain mismatch. FactCC was fine-tuned on CNN/DailyMail news summarisation, not on short factoid QA. Its consistency judgements on brief declarative answers (median 12–14 words) may be less reliable than on the multi-sentence summaries it was trained for.

Surface form mismatch. GraphRAG answers derive their vocabulary from verbalised triple labels. When ontological labels differ from the expected surface form (e.g. `Dentcheva-[field]->mathematical-optimisation` vs. *"Mathematical optimization"*), FactCC penalises the answer despite its semantic correctness.

FactAcc. We investigated **FactAcc** (the REBEL-based metric in MIRAGE) as a structurally fairer alternative, but found it unsuitable: REBEL [17] requires full sentence context to extract

`(subject, predicate, object)` triples, and our 1–3 word answers provide no syntactic structure. Empirical validation on a six-entry sample confirmed degenerate self-loops and a mean score of 0.055 with 5/6 entries at exactly zero.

SBERT as sanity check. We complement FactCC with Sentence-BERT cosine similarity between the generated answer and the ground-truth reference (§6.6). Its positive correlation with the corrected faithfulness ($r \approx +0.24$) confirms directional agreement, while the moderate magnitude reflects that the two measure distinct properties: context faithfulness vs. answer correctness.

6.3 Quantitative Results

Table 6 reports faithfulness scores computed on *answered* entries only (coverage errors excluded), alongside coverage error rates for all five pipelines. The parent-child ablation is reported separately in Section 5.6.

Table 6: FactCC faithfulness (coverage errors excluded), coverage error rate, hallucination rate, and SBERT correctness across the five pipelines. All faithfulness figures are computed on answered entries only.

Pipeline	Answered	Coverage err.	Mean faith.	Halluc.	SBERT
RAG baseline	453 / 547	17.2%	0.547	44.6%	0.446
GraphRAG baseline	364 / 547	33.5%	0.502	49.2%	0.410
RAG + CE	483 / 547	11.7%	**0.560**	**43.9%**	**0.464**
GraphRAG + CE	397 / 547	27.4%	0.499	49.4%	0.437
Graph-Chunk + CE	490 / 547	**10.4%**	0.527	47.1%	0.454

The ordering is led by RAG+CE, which attains the highest faithfulness (0.560) and the highest SBERT (0.464); the plain RAG baseline follows (0.547). The cross-encoder thus helps the text pipeline on every axis (faithfulness $\Delta = +0.013$, coverage 17.2 → 11.7%). On the graph side the picture differs: GraphRAG+CE lowers abstention (33.5 → 27.4%) and raises SBERT (+0.027) but its faithfulness is flat-to-slightly-lower than the GraphRAG baseline (0.499 vs. 0.502). Graph-Chunk+CE attains the lowest abstention of all pipelines (10.4%) and the best multi-hop faithfulness, but does not beat RAG+CE on any global metric.

6.4 Results by Question Type

Table 7 breaks down faithfulness by question type for the all five main pipelines.

The multi-hop column is the most informative. The GraphRAG baseline collapses on multi-hop questions (0.314, about 10 points below RAG), confirming the context-gap failure mode of §6.7. Both hybrids recover it: RAG+CE reaches the best multi-hop faithfulness of the study (0.438, $\Delta = +0.024$ over RAG), and Graph-Chunk+CE recovers most of GraphRAG's deficit (0.426, +0.113 over GraphRAG and above the plain RAG baseline). A two-hop chain that cosine similarity over individual triples cannot recover is bridged either by a reranked text chunk that mentions both entities (RAG+CE) or by the combined chunk-plus-triple context (Graph-Chunk+CE).

The Cross-Encoder reranker amplifies this effect: it can promote a single text chunk containing the full reasoning chain over several individually-relevant triples that each provide only one hop.

6.5 Numeric vs. Textual Answers

We classify a question's reference answer as *numeric* when it begins with a digit (years, counts, quantities), and *textual* otherwise. Results are shown in Table 8.

Table 7: FactCC mean faithfulness per question type (coverage errors excluded). Δ compares each pipeline to the RAG baseline.

Pipeline	Type	Mean faith.	Δ vs RAG
RAG baseline	T5	0.636	—
	Single-hop	0.524	—
	Multi-hop	0.414	—
GraphRAG	T5	0.610	−0.026
	Single-hop	0.493	−0.031
	Multi-hop	0.314	−0.101
RAG + CE	T5	**0.663**	+0.026
	Single-hop	**0.525**	+0.001
	Multi-hop	**0.438**	+0.024
GraphRAG + CE	T5	0.552	−0.085
	Single-hop	0.516	−0.008
	Multi-hop	0.342	−0.072
Graph-Chunk + CE	T5	0.614	−0.023
	Single-hop	0.499	−0.025
	Multi-hop	0.426	+0.012

Table 8: FactCC faithfulness split by answer type: numeric reference answers (beginning with a digit) vs. textual (coverage errors excluded).

Pipeline	Numeric	Textual	Δ
RAG baseline	0.628	0.535	+0.093
GraphRAG	0.555	0.495	+0.061
RAG + CE	0.632	0.550	+0.082
GraphRAG + CE	0.526	0.495	+0.032
Graph-Chunk + CE	0.603	0.516	+0.086

All five pipelines handle numeric answers better than textual ones (Δ between +0.03 and +0.09). The effect is present across paradigms: numerical values are stated verbatim in text chunks and encoded as objects in triples, so both retrievers reproduce them more reliably than free-form textual answers.

6.6 Answer Correctness: Sentence-BERT Cosine Similarity

We compute the cosine similarity between the LLM-generated answer and the ground-truth reference using `all-MiniLM-L6-v2`. This metric — denoted *SBERT score* — requires no source document and is immune to the surface-form confounds that affect FactCC. Table 9 reports results globally and by question type for the all five main pipelines.

RAG+CE achieves the best SBERT score globally (0.464) and on every question type, confirming on a source-independent metric that cross-encoder reranking on the text pipeline produces the most correct answers. Graph-Chunk+CE is second (0.454), still above the plain RAG baseline (0.446) and well above GraphRAG (0.410); on multi-hop it reaches 0.410 vs. 0.402 for RAG.

Two caveats apply to absolute SBERT scores. First, they are systematically low (0.39–0.49) because generated answers are full declarative sentences while ground-truth references are short extractive spans; the cosine is mechanically depressed by this verbosity asymmetry. Second, GraphRAG's lower score is partly driven by leaf-node abstentions that yield near-zero cosine similarity with any ground-truth span.

Table 9: Sentence-BERT cosine similarity (generated answer ↔ ground-truth), coverage errors excluded.

Pipeline	Type	Mean SBERT
RAG baseline	Global	0.446
	T5	0.479
	Single-hop	0.436
	Multi-hop	0.402
GraphRAG	Global	0.410
	T5	0.435
	Single-hop	0.403
	Multi-hop	0.379
RAG + CE	Global	**0.464**
	T5	**0.504**
	Single-hop	**0.449**
	Multi-hop	**0.418**
GraphRAG + CE	Global	0.437
	T5	0.461
	Single-hop	0.440
	Multi-hop	0.379
Graph-Chunk + CE	Global	0.454
	T5	0.492
	Single-hop	0.441
	Multi-hop	0.410

6.7 Qualitative Analysis of Failure Modes

Manual inspection of low-scoring entries (faithfulness < 0.3, coverage errors excluded) reveals four distinct failure modes.

Parametric memory leakage (RAG). Among the 194/453 critical RAG cases, a substantial fraction involve answers that are factually plausible but absent from any retrieved chunk. The LLM draws on its parametric knowledge rather than the provided context (e.g. *"Joe Biden"* for a leadership question whose retrieved passages do not name the current leader). FactCC captures this as low faithfulness because the source does not lexically support the claim, even though the answer may be externally correct.

Surface form mismatch (GraphRAG). GraphRAG answers derive their vocabulary from triple labels; when ontological labels diverge from natural text (e.g. `Dentcheva-[field]->mathematical-optimis`), FactCC penalises the answer despite its semantic correctness. The Cross-Encoder partially mitigates this by selecting triples whose verbalisation is closer to the question phrasing, explaining the SBERT gain of GraphRAG+CE ($\Delta = +0.027$) despite its faithfulness not improving ($\Delta = -0.004$).

Context gap (GraphRAG multi-hop). Cosine similarity over individual verbalised triples often recovers only the triple anchored at the entity most similar to the question, missing the intermediate relation required to complete a two-hop chain. Graph-Chunk+CE addresses this directly: text chunks containing both entities of a reasoning chain are retrieved by the dense chunk index and promoted by the Cross-Encoder.

Leaf-node abstention (GraphRAG). When the target entity has no outgoing edges, GraphRAG retrieves no relevant context and the LLM produces an honest abstention (*"the context does not contain information about X"*). These are classified as coverage errors and excluded from faithfulness computation (§6.1). The reduction in coverage error rate from 33.5% to 27.4% for GraphRAG+CE and to 10.4% for Graph-Chunk+CE reflects the reranker's ability to surface relevant text chunks even when graph retrieval alone would fail. As a structured

correlate of the a-priori taxonomy of Section 2, genuine *ontology violations* proved rare in practice: because GraphRAG answers are drawn directly from ontology-typed triples, they seldom assert type-incompatible relations, so the observed failures cluster in the four modes above.

6.8 Comparative Discussion

Overall ranking. By faithfulness the ordering is RAG+CE (0.560) > RAG (0.547) > Graph-Chunk+CE (0.527) > GraphRAG (0.502) > GraphRAG+CE (0.499). The SBERT metric gives the same top: RAG+CE (0.464) > Graph-Chunk+CE (0.454) > RAG (0.446) > GraphRAG+CE (0.437) > GraphRAG (0.410). The two metrics agree on the winner: RAG+CE, the text pipeline with hybrid recall and cross-encoder reranking.

The cross-encoder is the decisive intervention — on text. Adding the recall-and-rerank stage to the text pipeline improves it on every axis (faithfulness +0.013, SBERT +0.018, multi-hop +0.024, abstention 17.2 → 11.7%), whereas adding the same stage to the triple pipeline reduces abstention (33.5 → 27.4%) and raises SBERT (+0.027) but leaves faithfulness flat-to-lower (−0.004). The reranker needs lexical free-text passages to score; verbalised triples give it less to work with, so its benefit is concentrated on the text modality. In this benchmark, graph augmentation does not beat a well-tuned text RAG+CE.

What Graph-Chunk+CE still contributes. Graph-Chunk+CE is the strongest graph-augmented variant: it has the lowest abstention of all pipelines (10.4%) and recovers most of GraphRAG's multi-hop deficit (0.426, +0.113 over GraphRAG, above plain RAG). But it does not surpass RAG+CE on any global metric, so its value is specific: minimal abstention and a closed multi-hop gap, obtained by always presenting both a text-chunk and a triple view to the generator.

Methodological alignment bias. A confound favours text retrieval throughout: FactCC scores the answer against the original *text* sentence, and T5 questions are themselves derived from text, so text-chunk answers align with the scorer. This is visible in the T5 column, where RAG and RAG+CE lead and GraphRAG trails by 0.026. A strictly fair comparison would use exclusively KG-derived questions — GraphRAG's native setting — so the current mixed-provenance design could understate GraphRAG's performance on its own question type; we test this directly in Section 6.9 and find that RAG retains a statistically significant advantage even on graph-derived questions.

6.9 Statistical Significance and Question Provenance

The faithfulness gaps in Table 6 are small, so we test which are statistically meaningful. For each pair of pipelines we restrict to the questions *answered by both* (paired on the question identifier) and report the mean faithfulness difference with a 95% paired-bootstrap confidence interval (20,000 resamples) and a two-sided sign-flip permutation p-value (Table 10).

Table 10: Paired significance of faithfulness differences (items answered by both pipelines; 95% paired-bootstrap CI; sign-flip permutation p).

Comparison	**Δ faith.**	**95% CI**	p
RAG + CE vs RAG	+0.012	[−0.016, +0.041]	0.40
RAG + CE vs RAG (multi-hop)	+0.024	[−0.037, +0.087]	0.45
RAG vs GraphRAG	+0.055	[+0.014, +0.096]	0.009
GraphRAG + CE vs GraphRAG	−0.007	[−0.040, +0.024]	0.65
RAG + CE vs Graph-Chunk + CE	+0.028	[−0.002, +0.059]	0.071
Graph-Chunk + CE vs GraphRAG (multi-hop)	+0.081	[−0.015, +0.178]	0.10

Two conclusions follow. First, the headline gap between the two best pipelines, RAG+CE over RAG ($\Delta = +0.012$), is *not* significant ($p = 0.40$, and the CI spans 0): on this benchmark RAG+CE and RAG are statistically indistinguishable on faithfulness, so the cross-encoder's value on the text pipeline is better read as its consistent gain in abstention and answer correctness than as a faithfulness improvement. Second, the gap that *is* significant is between the text pipelines and GraphRAG (RAG vs GraphRAG $\Delta = +0.055$, $p < 0.01$); adding the cross-encoder to the triple pipeline leaves its faithfulness unchanged (GraphRAG+CE vs GraphRAG $\Delta = -0.007$, $p = 0.65$), confirming on a paired test that the reranker's benefit is concentrated on the text modality.

Question provenance. Because the question set mixes text-derived (T5) and graph-derived (single- and multi-hop) questions, we split faithfulness by provenance (Table 11). Every pipeline scores markedly higher on text-derived questions (by +0.09 to +0.18), confirming the alignment bias of Section 6.8: FactCC scores the answer against the source *text*, which favours text-derived questions. Crucially, the bias does not overturn the ranking: restricted to graph-derived questions — GraphRAG's native setting — RAG still significantly outperforms GraphRAG ($\Delta = +0.055$, 95% CI [+0.004, +0.106], $p = 0.036$, paired on 211 questions). The mixed-provenance design therefore inflates the absolute scores of the text pipelines but does not, by itself, account for GraphRAG's deficit.

Table 11: FactCC faithfulness by question provenance (answered only). Text-derived = T5 questions; graph-derived = single- and multi-hop questions.

Pipeline	Text-derived (T5)	Graph-derived	Δ
RAG baseline	0.636	0.481	+0.156
GraphRAG	0.610	0.434	+0.176
RAG + CE	0.663	0.489	+0.174
GraphRAG + CE	0.552	0.462	+0.089
Graph-Chunk + CE	0.614	0.469	+0.145

6.10 Pipeline Evolution: Performance Trajectory

Figure 2 plots the faithfulness and SBERT correctness scores across the five pipelines in the order they were implemented, illustrating how successive architectural choices affected each metric.

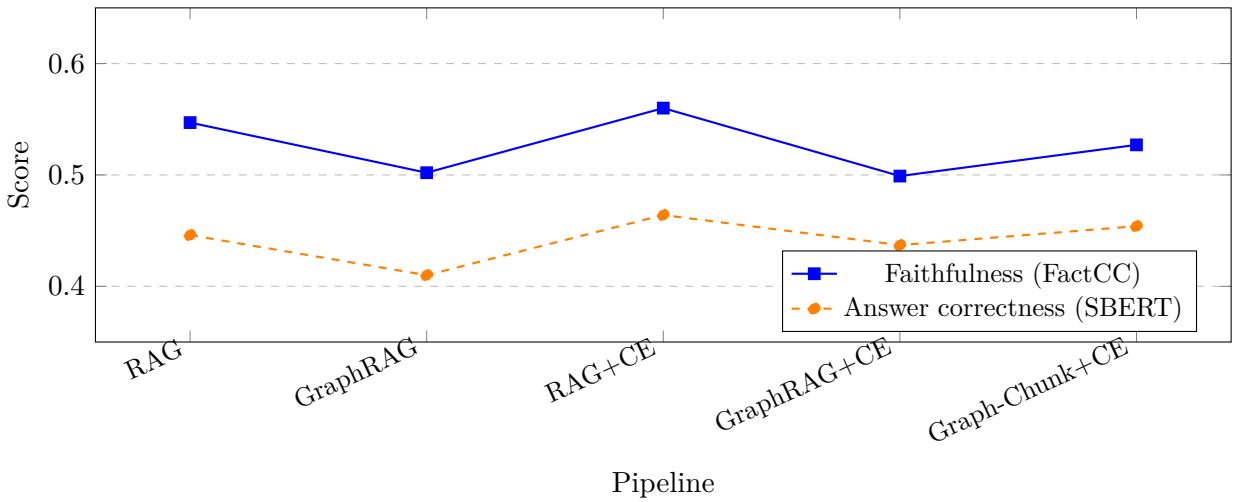


Figure 2: Faithfulness (FactCC, answered entries only) and answer correctness (Sentence-BERT) across the five pipelines in implementation order. RAG+CE is the best on both metrics; the graph-based pipelines trail.

Two observations stand out. First, the two metrics now agree on the winner (RAG+CE), confirming the cross-encoder's benefit to the text pipeline. Second, the trajectory is non-monotonic: GraphRAG drops below RAG, the cross-encoder lifts the text pipeline above both baselines (RAG+CE) but does not lift the triple pipeline (GraphRAG+CE stays at the GraphRAG level on faithfulness), and Graph-Chunk+CE lands between the graph and text pipelines.

6.11 Retrieval Depth Analysis

A natural question arising from the pipeline comparison is whether the observed differences are stable across retrieval depths, or whether a different value of k (the number of retrieved passages) would change the relative ranking. We therefore conducted a systematic **K-sweep** on a fixed 50-question *validation* subset, in two stages: a coarse grid $k \in \{2, 5, 10, 15, 20\}$ to reveal the broad trend, followed by a granular rerun (additional integer depths up to k=30 around each maximum). Figure 3 plots faithfulness as a function of k; for Graph-Chunk+CE, k denotes the per-leg budget ($k_{\text{graph}} = k_{\text{chunk}} = k$).

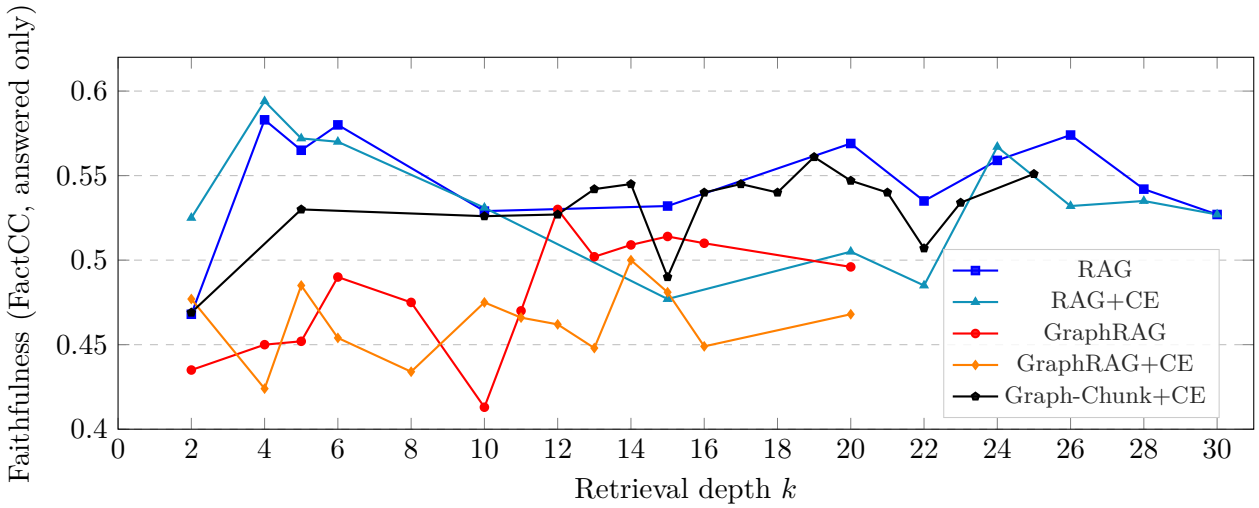


Figure 3: FactCC faithfulness (answered entries only) versus retrieval depth k for the five pipelines, measured on a 50-question validation subset. Text pipelines (RAG, RAG+CE) peak at small $k \approx 4$ and degrade as extra chunks dilute the context; graph pipelines (GraphRAG, GraphRAG+CE, Graph-Chunk+CE) climb to $k \approx 12$–20, where the atomic nature of triples requires a larger budget. The curves are noisy because of the small validation set, so we read trends rather than individual points.

Optimal k. The faithfulness-maximising depth differs sharply by paradigm: $k^\star = 4$ for both text pipelines (RAG, RAG+CE), $k^\star = 12$ for GraphRAG, $k^\star \approx 13$ for GraphRAG+CE, and $k^\star \approx 20$ per leg for Graph-Chunk+CE. These are the values frozen into the final pipelines (Table 4).

A text–graph asymmetry. Text pipelines peak early and then decline: beyond $k \approx 4$ each additional chunk adds more distracting than useful content, a "lost-in-the-middle" dilution effect (the same mechanism that explains the negative parent-child result of Section 5.6). Graph pipelines show the opposite trend, climbing until $k \approx 12$–20, because a verbalised triple carries a single atomic fact and many are needed to assemble enough evidence. Crucially, larger k also *lowers abstention* for graph pipelines: GraphRAG's coverage-error rate falls from $\approx 40\%$ at k=2 to $\approx 32\%$ at k=20, and Graph-Chunk reaches 0% on the validation subset for $k \geq 19$. For graph retrieval, depth therefore improves faithfulness and coverage simultaneously, up to the optimum.

Stability and protocol. Across essentially all k, the text-based pipelines dominate the graph-only pipelines on faithfulness and Graph-Chunk+CE tracks the text pipelines closely, so the ordering of Section 6.3 is not an artefact of a single k choice. Finally, the selected $k^\star$ were frozen on this validation subset and only then applied to the full 547-question test set used for all other results, so the depths are not overfit to the test data.

7 Conclusion

This project set out to answer one question: *how does the retrieval mechanism influence the frequency and nature of hallucinations in LLM-based question answering?* The empirical answer, across 547 question–answer pairs and five pipelines (plus a parent–child ablation) evaluated with FactCC and Sentence-BERT on two knowledge graph benchmarks, is nuanced: the retrieval paradigm determines not just the overall hallucination rate but the *type* of failure, and hybrid architectures can selectively address specific failure modes.

RAG+CE — the text pipeline with hybrid recall and cross-encoder reranking — achieves the highest mean faithfulness (0.560) and answer correctness (0.464), improving over the plain RAG baseline (0.547) on every axis; this faithfulness gain over RAG is not statistically significant (Section 6.9), whereas the gap to GraphRAG is. GraphRAG trails (0.502, 49.2% hallucination) due to three concurrent failure modes: surface-form mismatch between triple vocabulary and FactCC's text-based scoring, context gaps in cosine-similarity retrieval for multi-hop queries, and a high coverage error rate (33.5%) from leaf-node abstentions. Each failure mode is distinct and addressable.

The **Graph-Chunk + Cross-Encoder** pipeline is the strongest graph-augmented variant. By retrieving text chunks and verbalised triples through two independent cross-encoder–reranked retrievers and letting the generator fuse the two labelled sources, it attains the lowest abstention rate of all pipelines (10.4%) and recovers most of GraphRAG's multi-hop deficit (multi-hop faithfulness 0.426, +0.113 over the GraphRAG baseline and above the plain RAG baseline). It does not, however, surpass RAG+CE, which leads on both faithfulness and correctness: in this benchmark, a well-tuned text pipeline outperforms graph augmentation. A parent–child chunking variant, tested as an alternative way to widen the textual context, was neutral-to-negative (slightly lower faithfulness than flat RAG, slightly less abstention), reinforcing that what reduces hallucination is reranked, filtered evidence, not merely more surrounding text.

Three broader lessons emerge. **First**, the retrieval mechanism is a first-class variable for hallucination behaviour: the same generator model (Qwen3-30B), presented with different context, produces answers of dramatically different faithfulness. **Second**, the same component plays different roles depending on the modality: hybrid recall plus cross-encoder reranking improves the text pipeline (RAG+CE) on every axis, but on the triple pipeline (GraphRAG+CE) it only reduces abstention and improves correctness without raising faithfulness — the reranker needs free-text passages to exploit, and verbalised triples give it less leverage. **Third**, the correct application of evaluation metrics is consequential: our evaluation surfaced a score-sign inversion in the MIRAGE library ($P(\text{INCORRECT})$ exported as $P(\text{FAITHFUL})$), corrected via faithfulness $= 1 -$ score and validated by the sign flip of its Pearson correlation with SBERT (from ≈ -0.24 to $\approx +0.24$).

Limitations. FactCC was trained on news summarisation and operates on surface form; its per-instance reliability on short-answer QA over knowledge graphs carries the caveats discussed in §6.2. The SBERT correctness metric is systematically depressed by the verbosity asymmetry between generated answers (full sentences) and ground-truth spans (1–3 words) and should be treated as a relative ranking indicator. All results are produced by a single generator model (Qwen3-30B); isolating generator effects from retrieval effects would require a multi-model comparison.

Future work. Five extensions are motivated by the findings of this project and by the discussion during its defense.

First, a **stratified abstention analysis** separating leaf-node abstentions from genuine fabrication would sharpen the estimate of GraphRAG's true hallucination rate. The current protocol detects abstention by an explicit response-pattern matcher and excludes it from faithfulness; training a dedicated abstention classifier would make the separation more robust and allow a three-way breakdown (faithful / hallucinated / abstained).

Second, a **provenance-controlled comparison**. The mixed-provenance question set introduces an alignment bias that favours RAG on T5 (text-derived) questions, where FactCC's text source matches the retrieved chunks. Re-running the evaluation on exclusively KG-derived questions — GraphRAG's native setting — would yield a fairer estimate of the structural advantage of graph retrieval that the current design likely understates.

Third, and most structurally important for multi-hop questions, is **iterative retrieval with sub-question decomposition**. The context gap failure mode of §6.7 stems from attempting to answer a multi-hop question in a single retrieval pass. A more principled approach would use a dedicated LLM reasoning step to decompose the original question into an ordered sequence of atomic sub-questions, each answerable by a single-hop retrieval, then chain the sub-answers as context for the final generation. Variants of this strategy appear in IRCoT [20] and ReAct [21], and it would directly target the residual multi-hop gap that persists even with Graph-Chunk+CE. Implementing and evaluating it on the current 547-question benchmark is a direct next step.

Fourth, a **study of the fusion strategy in Graph-Chunk+CE**. The dual pipeline currently concatenates the reranked text-chunk block and the verbalised-triple block in a fixed order and under a fixed budget. The way the two `[CHUNK]` and `[KG]` blocks are concatenated and ordered may itself influence how the generator fuses them, independently of their content. Systematically varying this fusion (the order of the two blocks, interleaving them rather than concatenating, and the chunk-to-triple budget ratio) would separate the effect of how the evidence is *presented* from the effect of what it *contains*.

Fifth, the generation of **multi-sentence questions**. Every question in the current set is answerable from a single source entry, which is one reason the text pipelines saturate at a small retrieval depth (Section 6.11): the supporting evidence almost always fits in one or two chunks. Generating questions whose answer is spread across *several* source sentences would create a regime in which a larger k is genuinely useful, stressing the retrievers and giving the retrieval-depth analysis more discriminative power.

References

[1] J. Plu, O. Moreno Escobar, E. Trouillez, A. Gapin, P. Lisena, T. Ehrhart and R. Troncy, *Text2KGBench-LettrIA: A Refined Benchmark for Text2Graph Systems*, Joint Proceedings of KBC-LM and LM-KBC @ ISWC 2025. https://ceur-ws.org/Vol-4041/paper3.pdf

[2] D. Wang and M. Iwaihara, *OSKGC: A Benchmark for Ontology Schema-based Knowledge Graph Construction from Text*, Joint Proceedings of KBC-LM and LM-KBC @ ISWC 2025. https://ceur-ws.org/Vol-4041/paper1.pdf

[3] S. Patil (`valhalla`), *t5-small-qa-qg-hl: T5-small fine-tuned for answer-aware question generation on SQuAD*, Hugging Face model card, 2020. https://huggingface.co/valhalla/t5-small-qa-qg-hl

[4] S. Patil, *Question Generation using transformers*, GitHub repository, 2020. https://github.com/patil-suraj/question_generation

[5] Z. Li, Z. Cao, P. Li, Y. Zhong and S. Li, *Multi-Hop Question Generation with Knowledge Graph-Enhanced Language Model (KGEL)*, Applied Sciences, vol. 13, no. 9, art. 5765, 2023. https://www.mdpi.com/2076-3417/13/9/5765

[6] A. Naeiji et al., *Question Generation Using Sequence-to-Sequence Model with Semantic Role Labels (QGwSRL)*, EACL 2023. https://aclanthology.org/2023.eacl-main.207.pdf

[7] B. Vendeville, L. Ermakova, P. De Loor, and J. Kamps, *MIRAGE: A Framework for the Quantitative Evaluation of Hallucinations in Retrieval-Augmented Generation*, ACM, 2024. https://dl.acm.org/doi/10.1145/3746252.3761644 Software library: https://github.com/bVendeville/MIRAGE

[8] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

[9] T. Brown *et al.*, "Language Models are Few-Shot Learners," *NeurIPS*, 2020.

[10] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[11] M. Bachmann, *RapidFuzz: A fast string matching library for Python*, GitHub, 2021. https://github.com/maxbachmann/RapidFuzz

[12] Qwen Team, *Qwen3-30B-A3B-Instruct-2507: A Mixture-of-Experts Language Model*, Hugging Face model card, 2025. https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507

[13] LangChain, *LangChain: Building applications with LLMs through composability*, https://www.langchain.com, 2022.

[14] LangChain, *Parent Document Retriever*, LangChain Documentation, 2023. https://python.langchain.com/docs/how_to/parent_document_retriever/

[15] W. Kryściński, B. McCann, C. Xiong, and R. Socher, "Evaluating the factual consistency of abstractive text summarization," in *Proc. EMNLP*, 2020, pp. 9332–9346.

[16] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods," in *Proc. SIGIR*, 2009, pp. 758–759.

[17] P. L. H. Cabot and R. Navigli, "REBEL: Relation Extraction By End-to-end Language generation," in *Findings of EMNLP*, 2021, pp. 2370–2381.

[18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.

[19] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[20] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions (IRCoT)," in *Proc. ACL*, 2023. https://arxiv.org/abs/2212.10509

[21] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023. https://arxiv.org/abs/2210.03629

[22] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," 2024. https://arxiv.org/abs/2404.16130

[23] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, 2024. https://arxiv.org/abs/2306.08302